44. La principal ventaja de usar un diccionario de clases organizado por módulos es:

A) Que el código se vuelve más rápido.

B) Facilita la división del trabajo en el equipo de desarrollo y el mantenimiento.

C) Reduce el costo del servidor.

D) Elimina la necesidad de hacer pruebas.

45. Si estás diseñando la clase "Cafeteria" y le asignas un método "calcularVentasDelDia()", estás definiendo:

A) Un atributo de la clase.

B) Un comportamiento o método de la clase.

C) Un estereotipo de frontera.

D) Una relación de herencia.

Alumno: Araceli Cayetano Modesto Grupo: 5601 Materia: Fundamentos de ingeniería de software
Profesor: Juan Alberto Flores Gonzalez

LECTURA

Instrucciones para el alumno: Imprime este documento. Subraya las ideas principales con azul (conceptos), rojo (comandos de GitHub) y verde (aplicación al proyecto TESE). Lleva este documento impreso a la clase para las actividades prácticas.

1- La Crisis del Software en el Contexto Universitario

La Ingeniería de Software (IS) nace a finales de los años 60 debido a la "crisis del software", un término acuñado para describir la dificultad de desarrollar sistemas complejos que funcionaran correctamente y a tiempo. En el contexto de nuestra institución (TESE), nos enfrentamos a un microcosmos de esta crisis: la cafetería del plantel sufre de cuellos de botella durante los recesos. Las filas son largas, los pedidos se toman en papel y los alumnos pierden tiempo valioso de su descanso. Para resolver esto, no basta con "saber programar" en Flutter. Se requiere IS.

Un enfoque sistemático implica definir primero el problema (lentitud en el servicio), analizar quiénes interactúan (alumnos, personal de cafetería, administradores) y proponer una solución de software que sea escalable, mantenible y medible. El paradigma orientado a objetos (POO) es el estándar actual. Dart, el lenguaje que usaremos en Flutter, es un lenguaje puramente orientado a objetos. Esto significa que nuestra app de cafetería debe modelarse como un conjunto de objetos (Productos, Pedidos, Usuarios) que colaboran entre sí, en lugar de una lista de tareas aisladas.

El Ciclo de Vida y la Elección de Tecnologías

2. Etapas del Desarrollo aplicadas a Flutter

El ciclo de vida del software clásico (cascada) nos dice que debemos pasar por Análisis, Diseño, Código, Pruebas y Mantenimiento. Sin embargo, en proyectos ágiles y modernos como el que haremos, estas etapas son iterativas.

- **Análisis:** Entrevistar a la señora de la cafetería para saber qué productos son los más vendidos y cuáles se agotan primero.
- **Diseño:** Decidir que la app tendrá una pantalla de menú (Widget ListView), un carrito de compras (Clase Carrito) y un sistema de pago simulado.
- **Implementación:** Escribir el código en Dart.
- **Pruebas:** Asegurar que la app no se cierre si el usuario intenta comprar 0 cafés.

El uso de **Herramientas CASE** (Computer-Aided Software Engineering) es vital. En este curso, usaremos dos herramientas principales: Draw.io **o Lucidchart** (para diagramas UML de diseño) y **GitHub** (como herramienta de gestión de configuración y control de versiones). GitHub no solo almacena el código, sino que es nuestra bitácora de ingeniería: nos dirá quién hizo qué, cuándo y por qué, garantizando la integridad del proyecto de la cafetería.

Ingeniería de Requisitos - Elicitación

3. Descubriendo lo que la Cafetería necesita

La Ingeniería de Requisitos es la fase más crítica. Si construimos una app hermosa en Flutter pero que no resuelve el problema de la cafetería, hemos fracasado. La **Elicitación** es el arte de descubrir los requisitos. No podemos simplemente asumir qué quieren los usuarios.

- **Observación pasiva:** Pasar un recreo completo observando la fila. Notaremos que el 70% de los alumnos pide café y pan, y que el pago con efectivo es el que más tarda.
- **Entrevista estructurada:** Hablar con el administrador. Preguntar: "¿Cómo llevan el control de inventario actualmente?".
- **Cuestionario:** Enviar un Forms a los alumnos para preguntar: "¿Preferirías pedir desde tu celular y recoger, o hacer fila?".

  De esta elicitación, extraeremos **Requisitos Funcionales** (Ej. "El sistema debe permitir al alumno seleccionar la cantidad de productos") y **Requisitos No Funcionales** (Ej. "La app debe cargar el menú en menos de 3 segundos"). Un requisito no funcional es tan importante como uno funcional; de nada sirve que la app tenga carrito de compras si consume toda la batería del celular del alumno.

Modelado de Requisitos - Casos de Uso

4. Casos de Uso para la App de Cafetería

Los casos de uso describen cómo un actor (Usuario) interactúa con el sistema para lograr un objetivo. Para nuestro proyecto, el actor principal es el "Alumno", pero también existe el "Cajero" y el "Administrador".

Un caso de uso debe tener un **Flujo Principal** (el camino feliz) y **Flujos Alternativos** (errores o excepciones).

- *Ejemplo TESE:* Caso de uso "Realizar Pedido".
    - *Flujo Principal:* El alumno abre la app -> ve el menú -> agrega un café -> confirma -> paga virtualmente -> recibe ticket de recolección.
    - *Flujo Alternativo 1:* El alumno intenta agregar un producto, pero el sistema detecta que **se agotó el stock**. El sistema muestra un mensaje: "Lo sentimos, no hay pan dulce".
    - *Flujo Alternativo 2:* El pago virtual falla porque el alumno no tiene saldo. El sistema le ofrece recargar.

      Estos flujos alternativos son los que realmente diferencian a un ingeniero de software de un programador junior. Un junior solo programa el camino feliz; un ingeniero prevé los fallos y los documenta en los requisitos.

Arquitectura de Clases - El Patrón BCE

5. Del Requisito al Objeto en Flutter

Para diseñar la arquitectura de nuestro sistema, utilizaremos el patrón **BCE (Boundary-Control-Entity)**, un estándar en el análisis orientado a objetos.

- **Boundary (Frontera):** Son las clases que interactúan con el usuario. En Flutter, son los **Widgets** (pantallas, botones). Ejemplo: PantallaMenu, PantallaCarrito.
- **Entity (Entidad):** Son las clases que representan los datos persistentes del negocio. Ejemplo: Producto (tiene nombre, precio, id), Usuario (tiene matrícula, saldo), Pedido.
- **Control (Controlador):** Es el "cerebro". Recibe las peticiones del Boundary, ejecuta las reglas de negocio y actualiza las Entities. Ejemplo: GestorPedidos. Si el usuario presiona "Pagar", el Widget (Boundary) avisa al GestorPedidos (Control), quien verifica el saldo del Usuario (Entity) y resta el stock del Producto (Entity).

Esta separación es vital para Flutter. Si mezclas la lógica de negocio en los Widgets, tu código se volverá un espagueti imposible de mantener. Al usar BCE, puedes cambiar la interfaz visual sin tocar la lógica de pagos.

El Diccionario de Clases

6. Documentando las Clases en Dart

El diccionario de clases es el contrato de nuestro software. Es un documento que detalla qué atributos (variables) y métodos (funciones) tendrá cada clase.

- **Clase:** Producto (Estereotipo: Entity).
  - *Atributos:* id (String), nombre (String), precio (double), stock (int).
  - *Métodos:* reducirStock() (void), obtenerDetalles() (String).
- **Clase:** Pedido (Estereotipo: Entity).
  - *Atributos:* folio (String), fecha (DateTime), listaProductos (List<Producto>), total (double).
  - *Métodos:* calcularTotal() (double), agregarProducto(Producto p) (void).

    Es importante notar que, en Dart, definiremos estos tipos de datos explícitamente (String, double, int). El diccionario de clases nos permite saber exactamente qué debemos escribir en nuestro archivo modelo.dart antes de siquiera abrir el editor de código.

Diagramas de Secuencia

7. Modelando el Tiempo: El Diagrama de Secuencia

Mientras que el diagrama de clases muestra una foto estática de cómo se relacionan los objetos, el **Diagrama de Secuencia** muestra una película: cómo interactúan los objetos a lo largo del tiempo.

Para la app del TESE, dibujaremos el diagrama de secuencia del "Proceso de Pago".

1. El Alumno interactúa con el Widget Pago (Boundary).
2. El Widget Pago envía un mensaje al GestorPagos (Control): procesarPago(monto).
3. El GestorPagos consulta a la Entidad Usuario: verificarSaldo().
4. La Entidad Usuario responde con un booleano (true/false).
5. Si es true, el GestorPagos actualiza el Pedido (Entity) a estado "Pagado" y se lo notifica al Widget Pago para que muestre la pantalla de éxito. Este diagrama nos ayuda a detectar errores lógicos antes de programar. Por ejemplo: ¿Qué pasa si el GestorPagos verifica el saldo, pero mientras tanto el alumno cierra la app? Ese tipo de preguntas se responden modelando el diagrama de secuencia.

Herramientas CASE y GitHub

8. GitHub como Herramienta CASE de Colaboración

En el desarrollo profesional, no se trabaja solo. Se trabaja en equipo. Para asegurar que todos los integrantes del equipo del TESE contribuyan realmente al proyecto de la cafetería, usaremos **GitHub** como nuestro repositorio central. GitHub es más que una nube; es un sistema de control de versiones. Imagina que es una "máquina del tiempo" para tu código.

- **Repositorio (Repo):** La carpeta del proyecto en la nube.
- **Commit:** Una "foto" de los cambios. Cada commit tiene un mensaje obligatorio que explica qué se cambió.
- **Branch (Rama):** Una línea paralela de desarrollo. Para evitar que el código se rompa, cada alumno creará una rama para su función (ej. feature-carrito) y luego la unirá (merge) a la rama principal (main).

- **Pull Request (PR):** Una solicitud para integrar tu rama a la principal. Aquí el profesor o los compañeros revisan el código antes de aceptarlo. Esto evita que alguien suba código que haga crashear la app de todos.

Comandos Básicos de Git y GitHub

9. Operando la Máquina del Tiempo

Para subir el código del proyecto de la cafetería a GitHub, es necesario dominar la terminal. (Subraya estos comandos en rojo).

1. **Configurar identidad (solo una vez):**
   `git config --global user.name "Tu Nombre"`
   `git config --global user.email "tu.correo@tese.edu.mx"`
2. **Iniciar el repositorio local en la carpeta de Flutter:**
   `git init`
3. **Ver el estado de los archivos (qué está modificado):**
   `git status`
4. **Preparar los archivos para el commit (añadirlos al "escenario"):**
   `git add .` (el punto significa "todos los archivos").
5. **Guardar los cambios (Crear el commit):**
   `git commit -m "Agregada clase Producto y pantalla de menú en Flutter"`
6. **Vincular tu carpeta local con el repositorio remoto de GitHub:**
   `git remote add origin https://github.com/tu-usuario/proyecto-cafeteria.git`
7. **Subir los cambios a la nube:**
   `git push -u origin main`
8. **Crear una rama para trabajar una nueva función sin afectar a los demás:**
   `git checkout -b feature-pantalla-pago`

Métricas de Calidad y Estándares

10. Asegurando la Calidad en el TESE

El objetivo final de la materia es "alcanzar estándares y métricas de calidad". ¿Cómo medimos la calidad de nuestra app de cafetería?

- **Métrica de Código:** Cantidad de "Code Smells" (código mal escrito). Si en el diccionario de clases definimos que Producto tiene un precio double, no debemos pasarle un String en la interfaz.

- **Métrica de Proceso:** Frecuencia de commits en GitHub. Un equipo con solo un commit al final del semestre demuestra un proceso de baja calidad (todo lo hicieron a última hora). Un equipo con commits diarios demuestra un proceso saludable.
- **Estándar de Documentación:** Todo diagrama UML debe seguir el estándar de la OMG (Object Management Group). Todo caso de uso debe tener su flujo alternativo documentado.

  Al finalizar el curso, la app de cafetería no solo debe funcionar, sino que su código en GitHub debe ser limpio, modular, documentado y fácil de entender por cualquier otro ingeniero. Eso es la verdadera Ingeniería de Software.

EJERCICIOS PRÁCTICOS

Ejercicio 1: Elicitación y Requisitos (Basado en Páginas 3 y 4)

- Instrucción: Basado en el proyecto de la cafetería TESE, redacta 2 Requisitos Funcionales y 2 Requisitos No Funcionales. Asegúrate de que sean medibles y no ambiguos. Además, redacta un Flujo Alternativo para el caso de uso "Recargar Saldo" (¿Qué pasa si el servidor del banco no responde?).

Ejercicio 2: Modelado UML - Diagrama de Clases (Basado en Páginas 5 y 6)

- Instrucción: Utilizando la herramienta CASE de tu preferencia (Draw.io, Lucidchart o StarUML), crea el Diagrama de Clases para el módulo de "Pedidos" de la cafetería. Debe incluir obligatoriamente las clases Usuario (Entity), Producto (Entity), Pedido (Entity) y GestorPedidos (Control). Añade sus atributos principales y al menos un método por clase.

Ejercicio 3: El Diccionario de Clases en Dart (Basado en Páginas 6 y 10)

- Instrucción: Abre tu entorno de Flutter y crea un archivo llamado modelo_producto.dart. Basándote en tu diagrama del Ejercicio 2, escribe el código real en Dart para la clase Producto.

EJERCICIOS PRÁCTICOS

Ejercicio 1: Elicitación y Requisitos (Basado en Páginas 3 y 4)

Instrucción: Basado en el proyecto de la cafetería TESE, redacta 2 Requisitos Funcionales y 2 Requisitos No Funcionales. Asegúrate de que sean medibles y no ambiguos. Además, redacta un Flujo Alternativo para el caso de uso "Recargar Saldo" (¿Qué pasa si el servidor del banco no responde?).

Requisitos Funcionales:

1. El sistema debe calcular y actualizar automáticamente el costo total del pedido sumando el precio vigente de los productos cada vez que el usuario modifique las cantidades en el carrito (Basado en RF06 y RF08 del proyecto de la cafetería).
2. El sistema debe impedir que el alumno agregue al carrito un producto cuyo estado de disponibilidad sea falso en la base de datos, mostrando un mensaje de advertencia en pantalla (Basado en la regla de negocio del proyecto).

Requisitos No Funcionales:

1. La aplicación móvil debe cargar y mostrar la información del catálogo de productos y sus imágenes en un tiempo máximo de 3 segundos bajo condiciones normales de red (Criterio de rendimiento).
2. El sistema debe mantener la integridad de los datos en la base de datos, garantizando que ninguna solicitud de pedido se procese si ocurre una pérdida de conexión con el backend (Criterio de integridad).

Flujo Alternativo del Caso de Uso "Recargar Saldo":

- **Paso 1:** El alumno ingresa el monto a recargar en la aplicación y presiona "Confirmar".
- **Paso 2:** El sistema intenta conectarse con la plataforma del banco.
- **Paso 3:** El servidor del banco excede el tiempo límite de espera y no responde.
- **Paso 4:** El sistema de la cafetería cancela la transacción de forma segura, mantiene el saldo original del alumno intacto y muestra el mensaje: "Error de conexión bancaria, por favor intenta más tarde".

Ejercicio 2: Modelado UML - Diagrama de Clases (Basado en Páginas 5 y 6)

- Instrucción: Utilizando la herramienta CASE de tu preferencia (Draw.io, Lucidchart o StarUML), crea el Diagrama de Clases para el módulo de "Pedidos" de la cafetería. Debe incluir obligatoriamente las clases Usuario (Entity), Producto (Entity), Pedido (Entity) y GestorPedidos (Control). Añade sus atributos principales y al menos un método por clase.

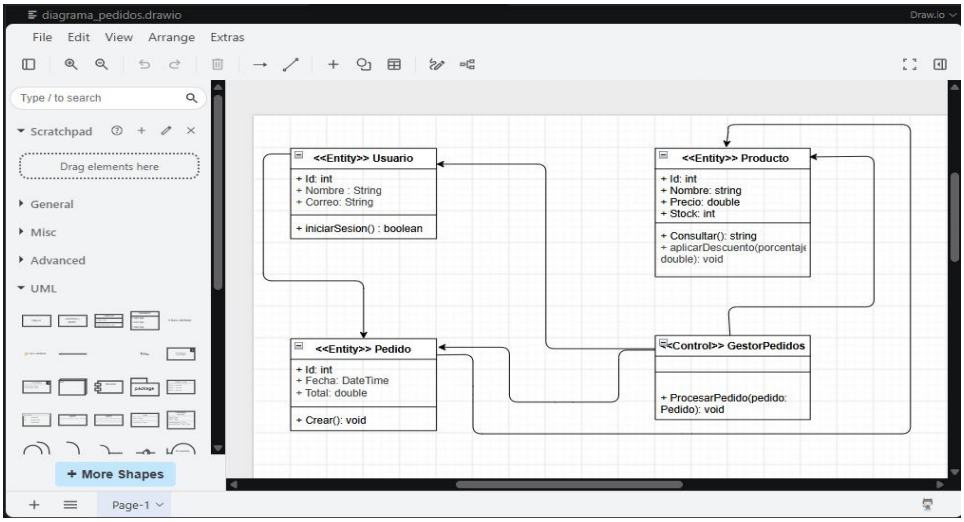


Figura 1: Diagrama de clases UML para el módulo de pedidos de la cafetería TESE, mostrando las clases *Usuario*, *Producto*, *Pedido* y *GestorPedidos* con sus atributos y métodos principales.

Ejercicio 3: El Diccionario de Clases en Dart (Basado en Páginas 6 y 10)

- Instrucción: Abre tu entorno de Flutter y crea un archivo llamado modelo_producto.dart. Basándote en tu diagrama del Ejercicio 2, escribe el código real en Dart para la clase Producto. Debe incluir sus atributos tipados (String, double, int) y un método llamado aplicarDescuento (double porcentaje) que reste el descuento al precio. Sube este archivo a tu repositorio de GitHub.

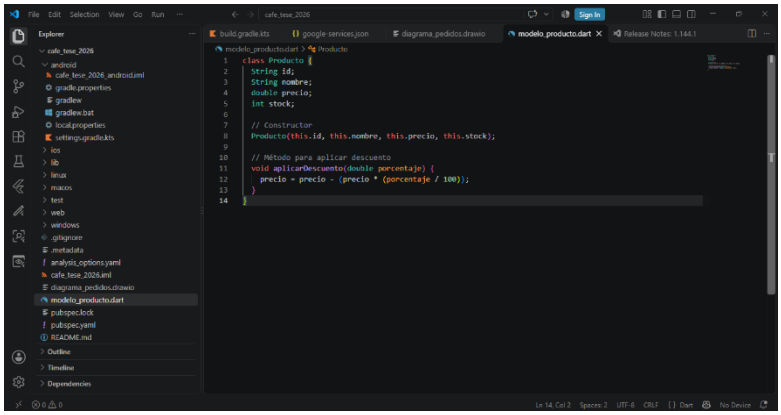


Figura 2: Implementación de la clase Producto en lenguaje Dart dentro del entorno de Visual Studio Code, incluyendo atributos tipados y el método aplicarDescuento

Ejercicio 4: Dominando GitHub (Basado en Páginas 8 y 9)

- Instrucción: (Trabajo en equipo). El líder del equipo debe haber creado el repositorio en GitHub. Cada miembro debe clonarlo (git clone [url]). Luego, cada miembro debe crear una rama con su nombre (git checkout -b rama [nombre]), crear un archivo de texto llamado tareas-[nombre].txt, escribir su aportación al proyecto, hacer un git add, un git commit -m "Aportación de [nombre]" y un git push origin rama-[nombre]. Finalmente, deben hacer un Pull Request en la interfaz web de GitHub.

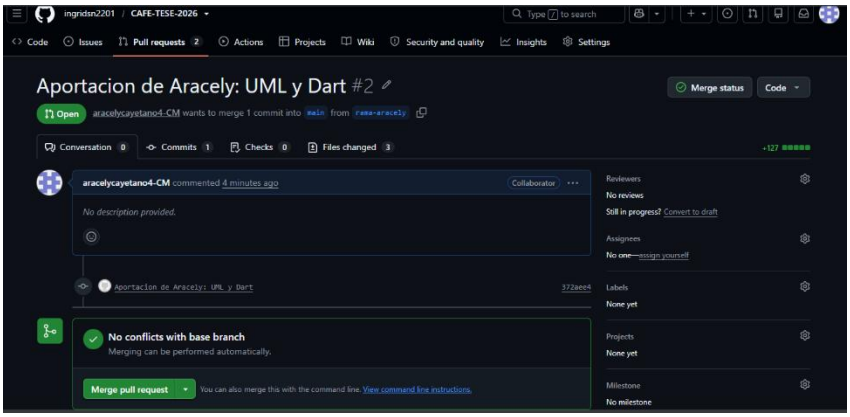


Figura 3: Interfaz de GitHub mostrando el estado del *Pull Request* para la integración de la rama personal (rama-aracely) hacia la rama principal (main)

Ejercicio 5: Diagrama de Secuencia (Basado en Página 7)

- Instrucción: Realiza un Diagrama de Secuencia UML que modele el proceso de "Agregar un producto al carrito". Los objetos participantes deben ser: Alumno (Actor), PantallaMenu (Boundary), GestorCarrito (Control) y Carrito (Entity). Debes mostrar al menos 4 mensajes síncronos (flechas) entre ellos, indicando la llamada a los métodos.

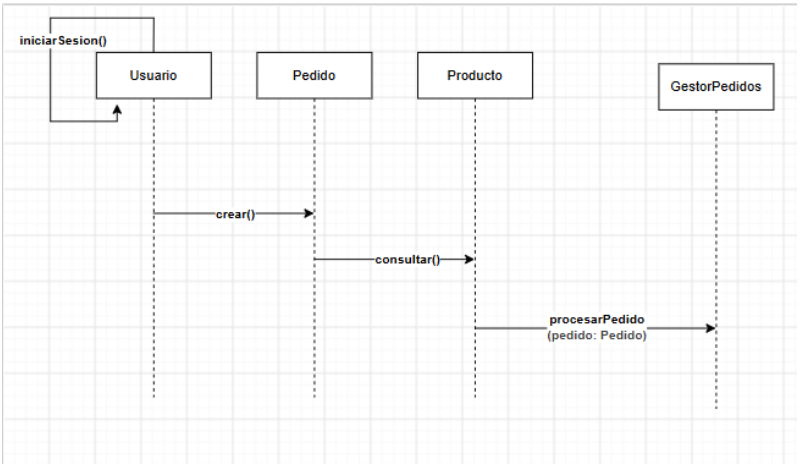


Figura 4: Diagrama de secuencia UML que modela el flujo de mensajes e interacciones síncronas entre los objetos participantes del sistema.